



MakersSoc Servos & Sequence

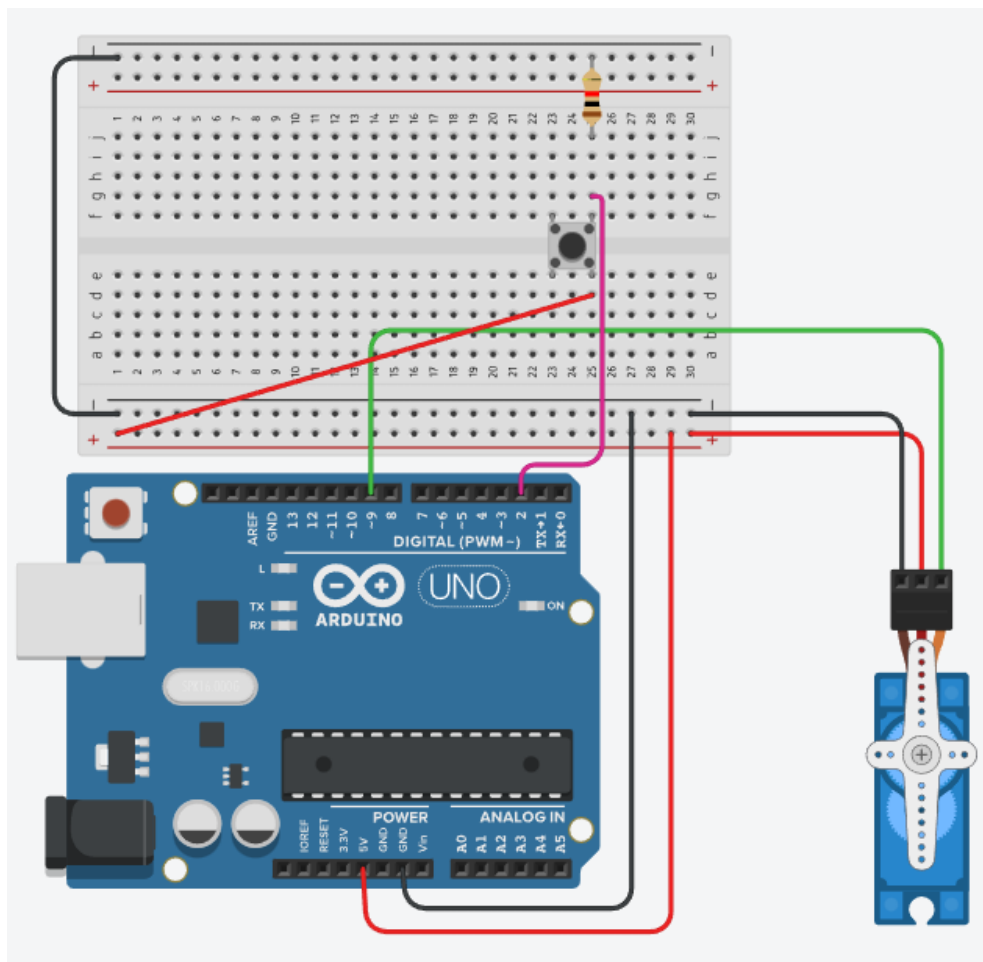
Olamide Akingbade, JJ Littleton

A servo motor is a small electrical device with rotating parts that can be moved to a particular angle, position and velocity. Today's workshop will teach us how to program and control our servo using an Arduino and how the order of instructions happens (known as sequence).

Equipment needed:

Arduino
USB B cable
Arduino IDE
(Personal) Toolkit
Breadboard
220Ω resistor
9g Micro Servo (SG90)
Wire

1. Assemble this circuit.



**2. Write the following set up code:**

```
#include <Servo.h>
int pos = 0;
Servo servo;
int servo_delay = 15;
int burst_internal_delay = 1000; //If this is too short, it will not have enough time to
reach positions

bool interrupted= false;

const byte servo_pin = 9;
const byte btnPin1 = 2;

int counter = 0;

void setup()
{
  servo.attach(servo_pin, 500, 2500);

  pinMode(btnPin1, INPUT);
  //attachInterrupt(digitalPinToInterrupt(btnPin1), btn1_pressed, RISING);
  Serial.begin(9600);
}
```

This set ups the variety of values that will be used now and later in the lab sheet.

The function “setup()” is called at the beginning of when the board can execute our code. This informs the device that a servo data pin is using the configured pin, in this case pin 9, as well as sets pin 1 to be an input pin.

3. Write the function “burst()” below “setup()”

```
//Instructions for servo - called when interrupt flag detected
void burst() {

  int burst_range = 40; //The angle range
  int burst_midpoint = 90; //The middle of the range
  int burst_upper = burst_midpoint + round(burst_range/2);
  int burst_lower = burst_midpoint - round(burst_range/2);

  //Tell servo to go to position in variable 'burst_midpoint'
  servo.write( burst_midpoint ); //mid
  Serial.println("mid"); //Print out can be viewed in the serial monitor
  delay(burst_internal_delay); // wait the set milliseconds for servo to reach the position
  servo.write( burst_upper ); //upper
  Serial.println("upper");
  delay(burst_internal_delay);
  servo.write( burst_lower ); //lower
  Serial.println("lower");
}
```



```
delay(burst_internal_delay);  
}
```

This function can be called in a variety of different places within the code. Meanwhile, only this part would need to be edited. Allows for easy repetition of code actually repeating lines of code.

4. Write the loop function

```
void loop()  
{  
    burst();  
}
```

This code is repeated once the device is set up.



Your code should look like this:

```
#include <Servo.h>
int pos = 0;
Servo servo;
int servo_delay = 15;
int burst_internal_delay = 1000; //If this is too short, it will not have enough time to reach positions

bool interrupted= false;

const byte servo_pin = 9;
const byte btnPin1 = 2;

int counter = 0;

void setup()
{
    servo.attach(servo_pin, 500, 2500);

    pinMode(btnPin1, INPUT);
    //attachInterrupt(digitalPinToInterrupt(btnPin1), btn1_pressed, RISING);
    Serial.begin(9600);
}

//Instructions for servo - called when interrupt flag detected
void burst() {

    int burst_range = 40; //The angle range
    int burst_midpoint = 90; //The middle of the range
    int burst_upper = burst_midpoint + round(burst_range/2);
    int burst_lower = burst_midpoint - round(burst_range/2);

    //Tell servo to go to position in variable 'burst_midpoint'
    servo.write( burst_midpoint ); //mid
    Serial.println("mid"); //Print out can be viewed in the serial monitor
    delay(burst_internal_delay); // wait the set milliseconds for servo to reach the position
    servo.write( burst_upper ); //upper
    Serial.println("upper");
    delay(burst_internal_delay);
    servo.write( burst_lower ); //lower
    Serial.println("lower");

    delay(burst_internal_delay);
}

void loop()
{
    burst();
}
```



5. Upload your code to the Arduino.

6. Next, write these two function between “burst()” and “loop()”

```
void btn1_pressed() // interrupt handler for button1
{
  Serial.println("Interrupt 1 (right button)"); //Immediately output upon interrupt, whereas
  flag may have a delay in detection
  interrupted= true;
}

//Checks interrupt flag & executes code
void interrupted_check() {
  if (interrupted== true) {

    burst();

    interrupted = false;
  }
}
```

If you have not already, add the following line to “setup()” before “Serial.begin(9600);”
attachInterrupt(digitalPinToInterrupt(btnPin1), btn1_pressed, RISING);

This line tells the board when the defined pin for the button is pressed, to call the function “btn1_pressed”.

The functions that were just written are the instructions to follow when the button is pressed. Simply printing to the serial menu and a separate function to check if a variable is true. If so, then call on “burst()”. After this, it will set the variable flag back to false. This is sequence, instructions performance one after each other.

An interrupt can be used to immediately perform new instruction while previous instructions are in sequence.

**7. Update "loop()" to sweep the full range of the servo**

```
void loop()
{
  Serial.print("Counter = ");
  Serial.println(counter);
  counter++;

  for (pos = 0; pos <= 180; pos += 1) {
    //Tell servo to go to position in variable 'pos'
    servo.write(pos);
    //Wait ms for servo to reach the position
    delay(servo_delay); //Wait
    interrupted_check();
  }
  for (pos = 180; pos >= 0; pos -= 1) {
    servo.write(pos);
    delay(servo_delay);
    interrupted_check();
  }
}
```

This loops through the from 0 to 180 degrees, asking the servo to move to these positions. At each degree, it is also checking if the interrupt flag is true and if so, then "burst()" will run due to it being in "interrupted_check()".

**8. Your final code should look like this:**

```
#include <Servo.h>
int pos = 0;
Servo servo;
int servo_delay = 15;
int burst_internal_delay = 1000; //If this is too short, it will not have enough time to
reach positions

bool interrupted= false;

const byte servo_pin = 9;
const byte btnPin1 = 2;

int counter = 0;

void setup()
{
  servo.attach(servo_pin, 500, 2500);

  pinMode(btnPin1, INPUT);
  attachInterrupt(digitalPinToInterrupt(btnPin1), btn1_pressed, RISING);
  Serial.begin(9600);
}

//Instructions for servo - called when interrupt flag detected
void burst() {

  int burst_range = 40; //The angle range
  int burst_midpoint = 90; //The middle of the range
  int burst_upper = burst_midpoint + round(burst_range/2);
  int burst_lower = burst_midpoint - round(burst_range/2);

  //Tell servo to go to position in variable 'burst_midpoint'
  servo.write( burst_midpoint ); //mid
  Serial.println("mid"); //Print out can be viewed in the serial monitor
  delay(burst_internal_delay); // wait the set milliseconds for servo to reach the position
  servo.write( burst_upper ); //upper
  Serial.println("upper");
  delay(burst_internal_delay);
  servo.write( burst_lower ); //lower
  Serial.println("lower");

  delay(burst_internal_delay);
}
```



```
void btn1_pressed() // interrupt handler for button1
{
  Serial.println("Interrupt 1 (right button)"); //Immediately output upon interrupt, whereas
  flag may have a delay in detection
  interrupted= true;
}

//Checks interrupt flag & executes code
void interrupted_check() {
  if (interrupted== true) {

    burst();

    interrupted= false;
  }
}

void loop()
{

  Serial.print("Counter = ");
  Serial.println(counter);
  counter++;

  for (pos = 0; pos <= 180; pos += 1) {
    //Tell servo to go to position in variable 'pos'
    servo.write(pos);
    //Wait ms for servo to reach the position
    delay(servo_delay); //Wait
    interrupted_check();
  }
  for (pos = 180; pos >= 0; pos -= 1) {
    servo.write(pos);
    delay(servo_delay);
    interrupted_check();
  }
}
```